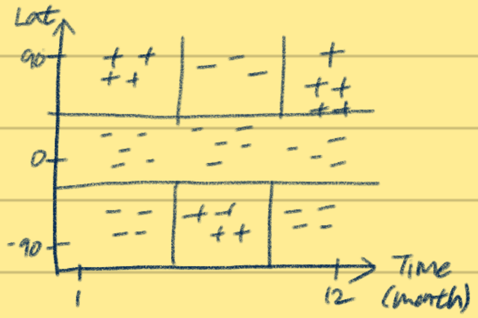
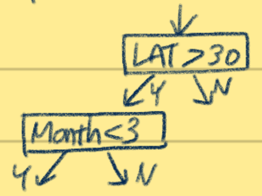
# Lec10 Decision trees and Ensemble methods

- Decision trees
- Ensemble methods
- Bagging
- Random Forest
- Boosting

## Decision Trees




Greedy, Top-down, Recursive Partitioning




Region $R_p$

Looking for a split $S_p$

$S_p(j,t) = (\underbrace{\{X \mid X_j < t,\ X \in R_p\}}_{R1}, \underbrace{\{X \mid X_j \geq t,\ X \in R_p\}}_{R2})$

How to choose splits? (What is useful to define a loss on a region?)

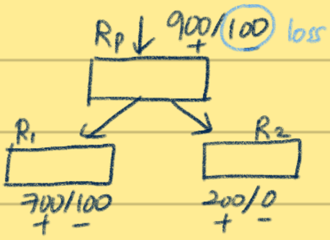
Define $L(R)$: loss on $R$

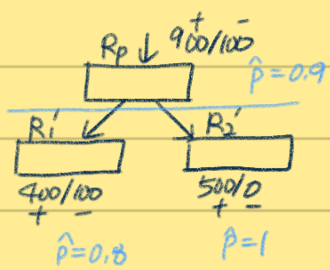
Given $C$ classes, define $\hat{p}_c$ to be the proportion of examples in $R$ that are of class $c$.

$L_{misclass} = 1 - \max_c \hat{p}_c$

$$\max_{j,t} \underbrace{L(R_p)}_{\substack{\text{parent loss} \\ \text{이미 정해짐}}} - \underbrace{(L(R_1) + L(R_2))}_{\substack{\text{children loss} \\ \text{Minimize this}}}$$

## Misclassification Loss has issues



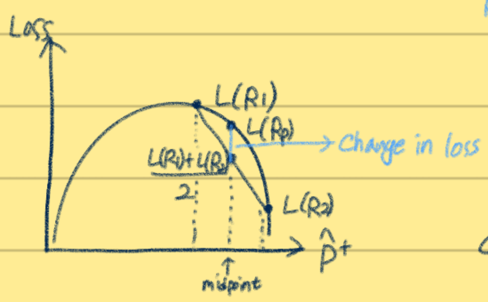




$L(R_1) + L(R_2) = 100 + 0 = 100$

$L(R_1') + L(R_2') = 100 + 0 = 100$

Instead, define cross-entropy loss (proportion)

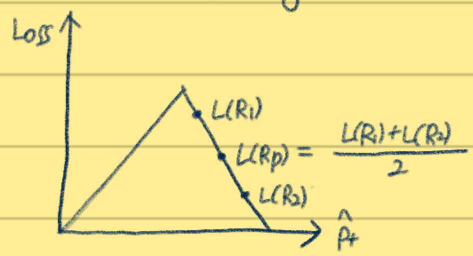
$L_{cross} = \sum_c \hat{p}_c \log_2 \hat{p}_c$




cross-entropy

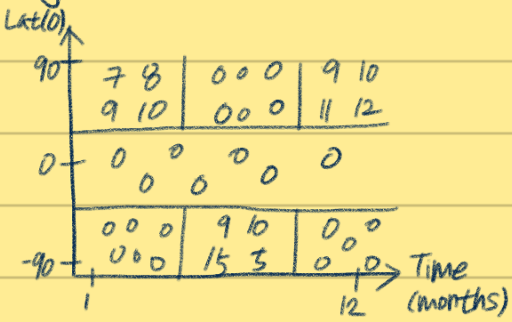
Gini

$\sum_c \hat{p}_c (1 - \hat{p}_c)$




Misclassification

## Regression Trees




Predict $\hat{y}_m = \frac{\sum_{i \in R_m} y_i}{|R_m|}$

$L_{square} = \frac{\sum_{i \in R_m} (y_i - \hat{y}_m)^2}{|R_m|}$

## Categorical Vars

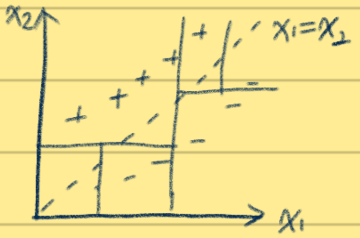
$\boxed{LOC \in \{N\}}$ q categories → $2^q$ possible splits

## Regularization of DTs

1) min leaf size
2) max depth
3) max number of nodes
4) min decrease in loss   Before split : $L(R_p)$ After split : $L(R_1) + L(R_2)$
5) pruning (misclassification with a validation set)

| Runtime | Test time | Train time |
|---|---|---|
| n examples | $O(d)$ | Each point is part of $O(d)$ nodes |
| f features | $d < \log_2 n$ | Cost of point at each node is $O(f)$ |
| d depth | | ∴ total cost is $O(nfd)$ |
| | | Data matrix is of size $n \cdot f$ |

## No additive structure




← DT는 성능이 떨어짐 선형회귀가 더 잘 맞음
when features are interacting additively with one another

## Recap

⊕ fast
⊕ easy to explain
⊕ interpretable
⊕ categorical vars

⊖ high variance
⊖ Bad at additive
⊖ low predictive accuracy (∵ the above two)
→ ensembling 하면 성능이 좋아짐

## Ensembling

Take $x_i$'s which are random variables (RV) that are independent identically distributed (iid)

$var(x_i) = \sigma^2$

$var(\bar{x}) = var(\frac{1}{n}\sum_i x_i) = \frac{\sigma^2}{n}$

Drop independence assumption so now $x_i$'s are i.d. $x_i$'s correlated by $\rho$

$var(\bar{x}) = \rho\sigma^2 + \frac{1-\rho}{n}\sigma^2$

## Ways to ensemble

1) different algorithms
2) different training sets
3) bagging (random forests)
4) boosting (Adaboost, xgboost)

## Bagging - Bootstrap Aggregation

Have a true population $P$

Training set $S \sim P$

Assume $P = S$

Bootstrap samples $Z \sim S$ (2/3 of S) $\rightarrow$ Bootstrap samples $Z_1, \ldots, Z_m$

Train model $G_m$ on $Z_m$

$$G_m = \frac{\sum_{m=1}^{M} G_m(X)}{M}$$

## Bias - Variance Analysis

$Var(\bar{x}) = \rho\sigma^2 + \frac{1-\rho}{M}\sigma^2$

Bootstraping is driving down $\rho$

More $M \rightarrow$ less variance

Bias slightly increased because of the random subsampling

## DTs + Bagging

DT are high variance, low bias

Ideal fit for bagging

## Random Forest

At each split, consider only a fraction of your total features.

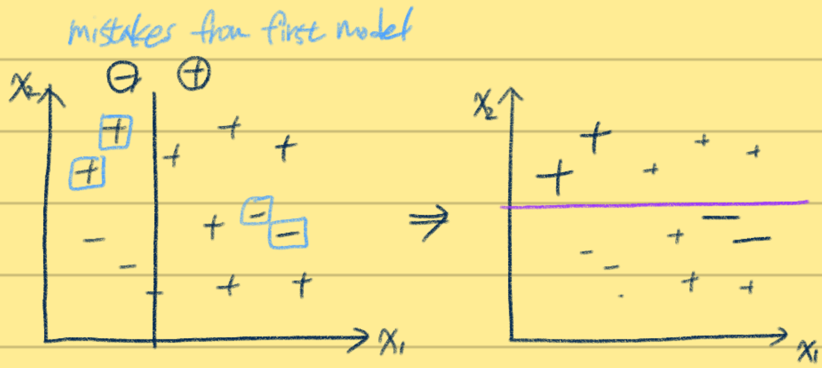
Decrease ρ

Decorrelate models

## Boosting

Decrease bias

Additive




<Adaboost Algorithm>

Determine for classifier $G_m$ a weight $\alpha_m$ proportional $\left(\frac{1-err_m}{err_m}\right)$

$G(x) = \sum_m \alpha_m G_m$ Each $G_m$ trained on re-weighted training set